

ECEC 661 – Final

Stack Processor 101 with new **ssq** (“stack the square”) instruction

Gary Pham (gp492)

June 3, 2026

1 Problem Statement

Starting from the assignment PDF’s Stack Processor 101 (**sp101**), implement a new machine instruction **ssq** – *stack the square* – which squares the value on top of the data stack in place. The user-visible calling convention is:

```
sc x ssq halt
```

For $x = 7$ the processor must leave 49 on the stack and the stack pointer must point to the next available address. Because **idle** initialises $sp = 128$ and **sc 7** writes $\text{mem}[128] = 7$ then bumps sp to 129, **ssq** overwrites $\text{mem}[128]$ with $7 \cdot 7 = 49$ and leaves $sp = 129$. Deliverables, scored from [Final/real/prob.md](#):

Part	Points	Artifact in this report
1. simulate correctness (sim wave)	10	Figs. 2–6
2. custom IP user logic (component + port map)	5	Listing 2
3.1 block diagram	3	Fig. 1
3.2 C test app	3	Listing 3
3.3 serial terminal snip (addr 128 = square)	4	Fig. 7 / Listing 4

2 Architecture

The IP keeps the assignment PDF’s three-block layout (Figure 1): an AXI4-Lite slave exposes the five software-visible registers, a **bus_ip_mem_bridge** multiplexes the bus or the processor onto the single port of **blk_mem_gen_0**, and the stack processor (**user_logic**) fetches/executes machine instructions out of that BRAM. Only the **exe** decoder of **user_logic** is extended with the new **ssq** opcode family; every other PDF instruction is unchanged.

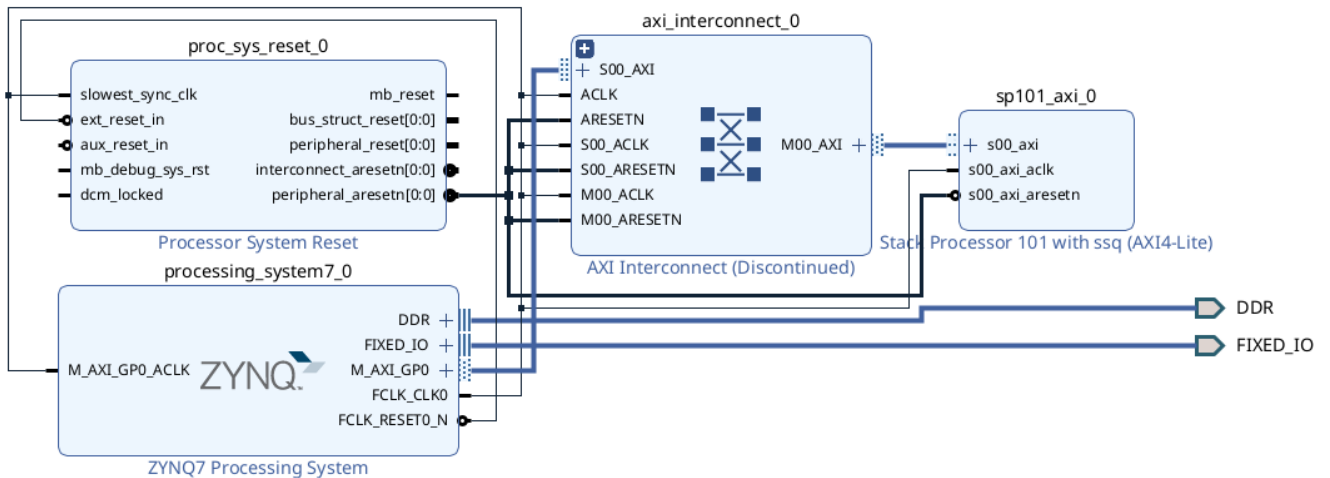


Figure 1: Vivado block design: ZYNQ7 Processing System, Processor System Reset, AXI Interconnect, and the custom **sp101_axi_0** IP (*Stack Processor 101 with ssq, AXI4-Lite*). **M_AXI_GP0** from the PS reaches the IP through the interconnect; **FCLK_CLK0** (100 MHz) and **FCLK_RESETO_N** drive the reset block. The IP is mapped at base 0x43C3_0000 (4KB) in the Address Editor.

3 Software Register Map

The slave maps five 32-bit registers into a 32-byte aperture, identical to the PDF’s reference C snippet:

Table 1: Software register map (byte offsets from XPAR_SP101_AXI_0_S00_AXI_BASEADDR).

Offset	Name	R/W	Purpose
0x00	slv_reg0	R/W	control: bit 0 =run, bit 1 =reset, bit 2 =bus2mem_en, bit 3 =bus2mem_we
0x04	slv_reg1	R/W	bus2mem_addr (lower 10 bits)
0x08	slv_reg2	R/W	bus2mem_data_in (32 bit)
0x0C	slv_reg3	R	sp2bus_data_out (32 bit)
0x10	slv_reg4	R	bit 0 = done flag

4 Instruction Set and ssq Opcode

The processor honours every opcode from the assignment PDF and adds a new family for **ssq**. The lowest hex digit of each constant is the micro-step index inside the instruction; the higher hex digits are the opcode group.

Table 2: Instruction set (opcode entry constants in [user_logic.vhd](#)).

Mnemonic	Code	Stack effect	Notes
halt	0x000000FF	–	raise done, halt
sc	0x00000001	push <i>c</i>	push constant pointed by pc
sl	0x00000011	pop <i>a</i> , push mem[<i>a</i>]	indirect load
ss	0x00000021	pop <i>d</i> , pop <i>a</i> , mem[<i>a</i>] ← <i>d</i>	indirect store
sadd	0x00000031	pop, pop, push sum	stack add
ssq	0x00000041	pop <i>x</i> , push $x \cdot x$	NEW – unary in place
scp	0x00000101	pop <i>s</i> , pop <i>d</i> , mem[<i>d</i>] ← mem[<i>s</i>]	single-word copy

The opcode 0x00000041 drops into the unused 0x4x slot above **sadd**, so none of the existing decoders are touched. Because **ssq** is unary and reuses the same stack slot for input and output, the stack pointer is unchanged: one “pop” and one “push” cancel.

4.1 ssq micro-program

The FSM walks **ssq** through six micro-states. States SSQ–SSQ4 schedule the BRAM read of mem[*sp* – 1] and burn the 4-cycle read latency (BRAM + mem_data_out_pre + mem_data_out). State SSQ5 captures *x* in temp1; state SSQ6 issues the write of $x \cdot x$ back to the same address and returns to fetch.

```

1  when SSQ =>                                -- start read of mem[sp-1]
2      mem_addr <= std_logic_vector(unsigned(sp) - 1);
3      we      <= "0"; ir <= SSQ2;
4  when SSQ2 => we <= "0"; ir <= SSQ3;          -- BRAM read latency
5  when SSQ3 => we <= "0"; ir <= SSQ4;          -- BRAM read latency
6  when SSQ4 => we <= "0"; ir <= SSQ5;          -- BRAM read latency
7  when SSQ5 =>                                -- x is now valid
8      temp1 <= mem_data_out;
9      we      <= "0"; ir <= SSQ6;
10 when SSQ6 =>                                -- write x*x back to mem[sp-1]
11     mem_addr  <= std_logic_vector(unsigned(sp) - 1);
12     mem_data_in <= std_logic_vector(
13         resize(unsigned(temp1) *
14             unsigned(temp1), 32));
15     we        <= "1";
16     n_s       <= fetch;

```

Listing 1: Core micro-states (excerpt of when SSQ ... body in [user_logic.vhd](#)).

Multiplication uses `numeric_std (unsigned × unsigned)` and `resize(...,32)` truncates the 64-bit product to the low 32 bits. All test vectors used here have $x \leq 46340$, so $x \cdot x < 2^{31}$ and no truncation occurs.

5 Part 2 – User Logic Instantiation (5 pts)

The required component declaration and port map live inside the AXI4-Lite slave wrapper [sp101_axi_v1_0_S00_AXI.vhd](#) (lines ~ 101–151). Control bits are sliced straight from `slv_reg0`; `bus2mem_addr`

takes the lower 10 bits of `slv_reg1`; `bus2mem_data_in` is the full 32 bits of `slv_reg2`; `sp2bus_data_out` and `done` feed the read mux (`slv_reg3` and `slv_reg4`).

```

1  -- Component declaration
2  component user_logic
3      generic (
4          A : natural := 10
5      );
6      port (
7          run          : in  std_logic;
8          reset        : in  std_logic;
9          bus2mem_en   : in  std_logic;
10         bus2mem_we    : in  std_logic;
11         ck            : in  std_logic;
12         bus2mem_addr  : in  std_logic_vector(A-1 downto 0);
13         bus2mem_data_in : in  std_logic_vector(31 downto 0);
14         sp2bus_data_out : out std_logic_vector(31 downto 0);
15         done          : out std_logic
16     );
17 end component;
18
19 -- Port map
20 U : user_logic
21     generic map (
22         A => 10
23     )
24     port map (
25         run          => slv_reg0(0),
26         reset        => slv_reg0(1),
27         bus2mem_en   => slv_reg0(2),
28         bus2mem_we    => slv_reg0(3),
29         ck            => S_AXI_ACLK,
30         bus2mem_addr  => slv_reg1(9 downto 0),
31         bus2mem_data_in => slv_reg2,
32         sp2bus_data_out => core_sp2bus_data_out,
33         done          => core_done
34     );

```

Listing 2: Component declaration and port map of the `user_logic` core inside `sp101_axi_v1.0_S00_AXI.vhd`.

6 Part 1 – Behavioral Simulation (10 pts)

`tb/tb_user_logic.vhd` drives the same `run`, `reset`, `bus2mem_*` signals the assignment’s xsim script forces, looping over a single vector list `SSQ_X_VALUES` and asserting that `mem[128]` holds $x \cdot x$ after each run. The vector list (Table 3) includes the prompt example ($x = 7$), three corner cases $\{0, 1, 2\}$, small/medium positive values, and two large operands that stress the 32-bit datapath.

Table 3: Test vectors driven by `tb_user_logic` (and by the on-board C app). All execute the program `sc x ssq halt` and verify `mem[128] = x2`.

x	x^2	note
7	49	prompt example
0	0	zero edge case
1	1	identity edge case
2	4	small
3	9	small
10	100	round decade
15	225	medium
16	256	power of two
255	65025	byte max
1000	1000000	round thousand
32767	1073676289	15-bit max
46340	2147395600	near $\sqrt{2^{31}}$

The wave script `scripts/sim_waves.tcl` sets the data-path radices to *unsigned* so the values read directly as decimal (much easier to confirm $7 \rightarrow 49$ than $0x7 \rightarrow 0x31$). Opcode signals like `ir` stay hex because they are encoded constants, not numbers.

Figures 2–4 show the full xsim run spanning all twelve scenarios; the integer `checks` counter climbs $0 \rightarrow 12$ while `errors` stays at 0. The `ssq_datapath` group in Figure 4 is the clearest proof of functional correctness: for every scenario the wave shows $\text{temp1} = x$, $\text{mem_data_in} = x^2$ on the SSQ6 write edge, and $\text{mem_data_out} = x^2$ on the post-run read-back through the bus. Figures 5 and 6 reproduce the Tcl-console transcript, listing the twelve [PASS] notes with both decimal and hex values and the TESTBENCH PASSED banner with `checks executed : 12, errors : 0`.

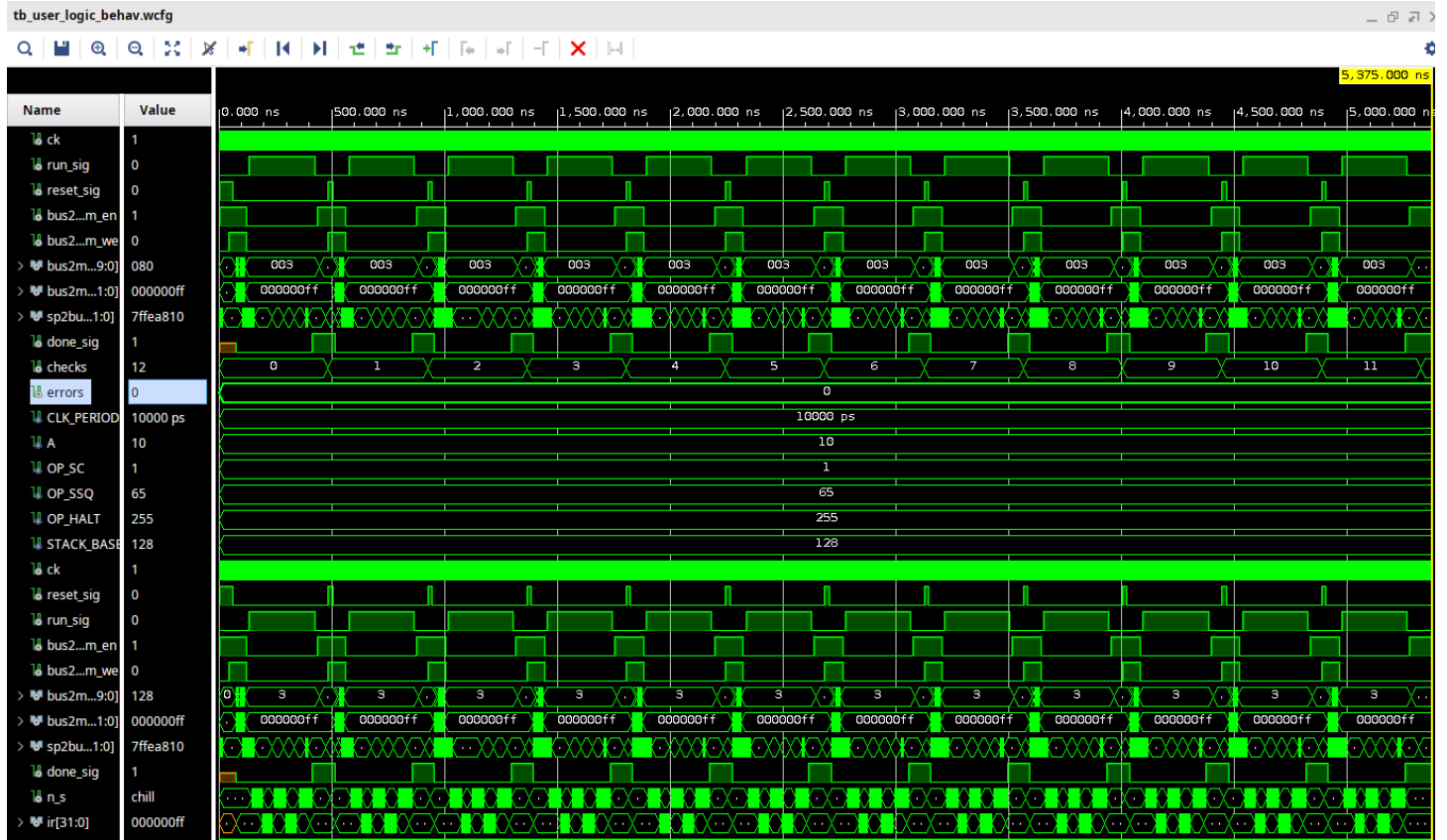


Figure 2: xsim waveform, overview. Top group shows `ck`, `run_sig`, `reset_sig`, `bus2mem.en/we`; the bus group shows `bus2mem.addr`, `bus2mem.data_in` and `sp2bus.data_out` in decimal. The `checks` integer (third pane from the top of the middle group) walks $0, 1, \dots, 11$ across the run; the `errors` integer is constant at 0.

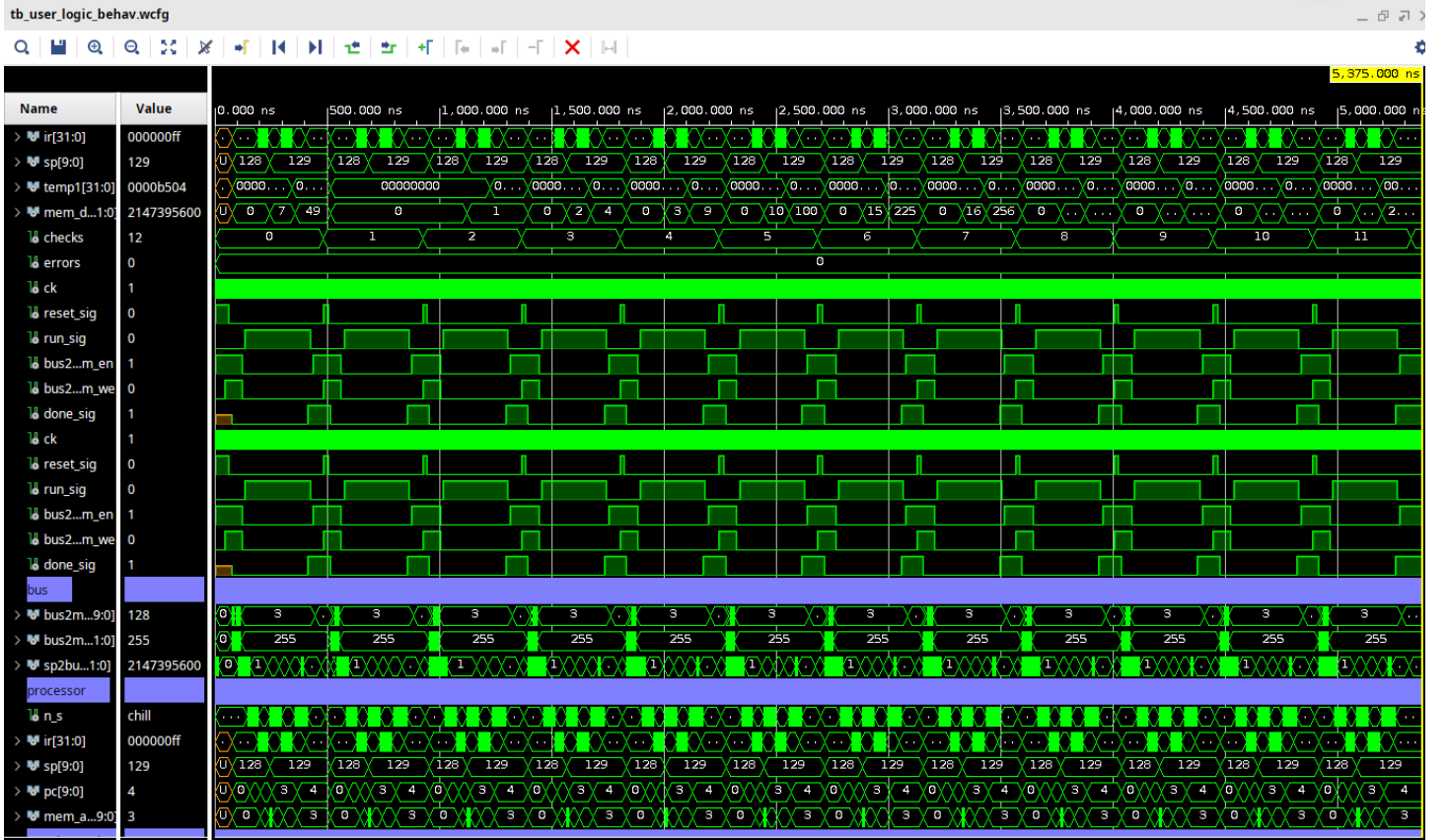


Figure 3: xsim waveform, processor view. *ir* (hex) walks through SC → SC2 → ... → SC5 → SSQ → SSQ2 → ... → SSQ6 → HALT every scenario. *sp* bounces 128 → 129 once per scenario (*sc* bumps it, *ssq* leaves it). *temp1* (decimal) holds the operand x and *mem_data_in* (decimal) carries x^2 into the BRAM on the SSQ6 write edge.

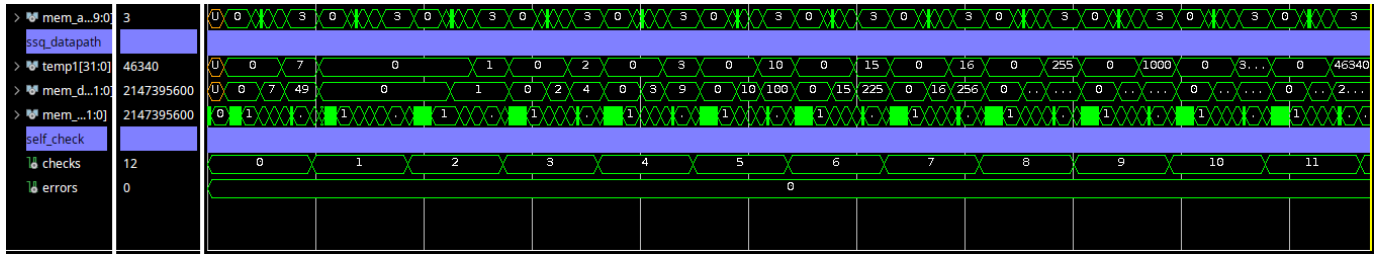


Figure 4: *ssq_datapath* zoom (decimal radix). Reading left to right the testbench drives $x = 0, 7, 1, 2, 3, 10, 15, 16, 255, 1000, 32767, 46340$; each *temp1* sample is followed one cycle later by the matching *mem_data_in* = x^2 and (after the read-back) *mem_data_out* = x^2 . The *checks* counter climbs $0 \rightarrow 12$ and *errors* stays 0, proving every vector matches.

Tcl Console

```

Block Memory Generator module tb_user_logic.dut.mm.inst.native_mem_module.blk_mem_gen_v8_4_12_inst is using a behavioral model for simulation which will not precisely model memory collision
Note: ===== Scenario: ssq x=7 -> 49
Time: 45 ns Iteration: 1 Process: /tb_user_logic/p_stim File: /home/kagimirudo/ECE661/Final/real/stackprocessor_101/tb/tb_user_logic.vhd
Note: [PASS] ssq x=7 -> 49 addr=128 got=49 expected=49 (0x00000031)
Time: 485 ns Iteration: 1 Process: /tb_user_logic/p_stim File: /home/kagimirudo/ECE661/Final/real/stackprocessor_101/tb/tb_user_logic.vhd
Note: ===== Scenario: ssq x=0 -> 0
Time: 485 ns Iteration: 1 Process: /tb_user_logic/p_stim File: /home/kagimirudo/ECE661/Final/real/stackprocessor_101/tb/tb_user_logic.vhd
Note: [PASS] ssq x=0 -> 0 addr=128 got=0 expected=0 (0x00000000)
Time: 925 ns Iteration: 1 Process: /tb_user_logic/p_stim File: /home/kagimirudo/ECE661/Final/real/stackprocessor_101/tb/tb_user_logic.vhd
Note: ===== Scenario: ssq x=1 -> 1
Time: 925 ns Iteration: 1 Process: /tb_user_logic/p_stim File: /home/kagimirudo/ECE661/Final/real/stackprocessor_101/tb/tb_user_logic.vhd
Note: [PASS] ssq x=1 -> 1 addr=128 got=1 expected=1 (0x00000001)
Time: 1365 ns Iteration: 1 Process: /tb_user_logic/p_stim File: /home/kagimirudo/ECE661/Final/real/stackprocessor_101/tb/tb_user_logic.vhd
Note: ===== Scenario: ssq x=2 -> 4
Time: 1365 ns Iteration: 1 Process: /tb_user_logic/p_stim File: /home/kagimirudo/ECE661/Final/real/stackprocessor_101/tb/tb_user_logic.vhd
Note: [PASS] ssq x=2 -> 4 addr=128 got=4 expected=4 (0x00000004)
Time: 1805 ns Iteration: 1 Process: /tb_user_logic/p_stim File: /home/kagimirudo/ECE661/Final/real/stackprocessor_101/tb/tb_user_logic.vhd
Note: ===== Scenario: ssq x=3 -> 9
Time: 1805 ns Iteration: 1 Process: /tb_user_logic/p_stim File: /home/kagimirudo/ECE661/Final/real/stackprocessor_101/tb/tb_user_logic.vhd
Note: [PASS] ssq x=3 -> 9 addr=128 got=9 expected=9 (0x00000009)
Time: 2245 ns Iteration: 1 Process: /tb_user_logic/p_stim File: /home/kagimirudo/ECE661/Final/real/stackprocessor_101/tb/tb_user_logic.vhd
Note: ===== Scenario: ssq x=10 -> 100
Time: 2245 ns Iteration: 1 Process: /tb_user_logic/p_stim File: /home/kagimirudo/ECE661/Final/real/stackprocessor_101/tb/tb_user_logic.vhd
Note: [PASS] ssq x=10 -> 100 addr=128 got=100 expected=100 (0x00000064)
Time: 2685 ns Iteration: 1 Process: /tb_user_logic/p_stim File: /home/kagimirudo/ECE661/Final/real/stackprocessor_101/tb/tb_user_logic.vhd
Note: ===== Scenario: ssq x=15 -> 225
Time: 2685 ns Iteration: 1 Process: /tb_user_logic/p_stim File: /home/kagimirudo/ECE661/Final/real/stackprocessor_101/tb/tb_user_logic.vhd
Note: [PASS] ssq x=15 -> 225 addr=128 got=225 expected=225 (0x000000E1)
Time: 3125 ns Iteration: 1 Process: /tb_user_logic/p_stim File: /home/kagimirudo/ECE661/Final/real/stackprocessor_101/tb/tb_user_logic.vhd
Note: ===== Scenario: ssq x=16 -> 256
Time: 3125 ns Iteration: 1 Process: /tb_user_logic/p_stim File: /home/kagimirudo/ECE661/Final/real/stackprocessor_101/tb/tb_user_logic.vhd
Note: [PASS] ssq x=16 -> 256 addr=128 got=256 expected=256 (0x00000100)
Time: 3565 ns Iteration: 1 Process: /tb_user_logic/p_stim File: /home/kagimirudo/ECE661/Final/real/stackprocessor_101/tb/tb_user_logic.vhd
Note: ===== Scenario: ssq x=255 -> 65025
Time: 3565 ns Iteration: 1 Process: /tb_user_logic/p_stim File: /home/kagimirudo/ECE661/Final/real/stackprocessor_101/tb/tb_user_logic.vhd
Note: [PASS] ssq x=255 -> 65025 addr=128 got=65025 expected=65025 (0x0000FE01)
Time: 4005 ns Iteration: 1 Process: /tb_user_logic/p_stim File: /home/kagimirudo/ECE661/Final/real/stackprocessor_101/tb/tb_user_logic.vhd
Note: ===== Scenario: ssq x=1000 -> 1000000
Time: 4005 ns Iteration: 1 Process: /tb_user_logic/p_stim File: /home/kagimirudo/ECE661/Final/real/stackprocessor_101/tb/tb_user_logic.vhd
Note: [PASS] ssq x=1000 -> 1000000 addr=128 got=1000000 expected=1000000 (0x000F4240)
Time: 4445 ns Iteration: 1 Process: /tb_user_logic/p_stim File: /home/kagimirudo/ECE661/Final/real/stackprocessor_101/tb/tb_user_logic.vhd
Note: ===== Scenario: ssq x=32767 -> 1073676289
Time: 4445 ns Iteration: 1 Process: /tb_user_logic/p_stim File: /home/kagimirudo/ECE661/Final/real/stackprocessor_101/tb/tb_user_logic.vhd
Note: [PASS] ssq x=32767 -> 1073676289 addr=128 got=1073676289 expected=1073676289 (0x3FFF0001)
Time: 4885 ns Iteration: 1 Process: /tb_user_logic/p_stim File: /home/kagimirudo/ECE661/Final/real/stackprocessor_101/tb/tb_user_logic.vhd
Note: ===== Scenario: ssq x=46340 -> 2147395600
Time: 4885 ns Iteration: 1 Process: /tb_user_logic/p_stim File: /home/kagimirudo/ECE661/Final/real/stackprocessor_101/tb/tb_user_logic.vhd
Note: [PASS] ssq x=46340 -> 2147395600 addr=128 got=2147395600 expected=2147395600 (0x7FEEA810)
Time: 5325 ns Iteration: 1 Process: /tb_user_logic/p_stim File: /home/kagimirudo/ECE661/Final/real/stackprocessor_101/tb/tb_user_logic.vhd

```

Figure 5: xsim Tcl console output for all twelve scenarios. Every vector reports [PASS] ssq x=X -> X^2 addr=128 got= X^2 expected= X^2 (0x...).

```

Note:
+-----+
|      spl01 + ssq stack-the-square TB summary      |
+-----+
checks executed : 12
errors          : 0
+-----+
Time: 5375 ns Iteration: 0 Process: /tb_user_logic/p_stim File: /home/kagimirudo/ECE661/Final/real/stackprocessor_101/tb/tb_user_logic.vhd
Note:
#####
##          TESTBENCH PASSED - all cases ok          ##
##          #####                                     ##
#####
Time: 5375 ns Iteration: 0 Process: /tb_user_logic/p_stim File: /home/kagimirudo/ECE661/Final/real/stackprocessor_101/tb/tb_user_logic.vhd
save_wave_config {/home/kagimirudo/ECE661/Final/real/spl01/tb_user_logic_behav.wcfg}

```

Figure 6: Final tb_user_logic banner. checks executed : 12, errors : 0, TESTBENCH PASSED -- all cases ok.

7 Part 3.2 – Bare-metal C Application (3 pts)

The bare-metal application `sw/main.c` drives the AXI4-Lite peripheral through the exact handshake described in the PDF and runs the same twelve vectors as the simulator (so the on-board UART log matches the simulator output line-for-line). The key driver helpers are reproduced in Listing 3; note the “never assert CTRL_RESET during mem_read” rule, which is the failure mode that silently masked the dump in early bring-up.

```

1 /* slv_reg0 control bits */
2 #define CTRL_RUN      (1U << 0)
3 #define CTRL_RESET    (1U << 1)
4 #define CTRL_BUS_EN   (1U << 2)
5 #define CTRL_BUS_WE   (1U << 3)
6
7 /* IMPORTANT: never assert CTRL_RESET while reading. Pulse reset only
8  * in processor_reset(); use plain CTRL_BUS_EN (+CTRL_BUS_WE for
9  * writes) for normal bus access. */
10 static void mem_write(u32 addr, u32 data)
11 {

```



```

12     SP101_mWriteReg(BASEADDR, REG_CTRL,      CTRL_BUS_EN | CTRL_BUS_WE);
13     SP101_mWriteReg(BASEADDR, REG_BUS_ADDR, addr & 0x3FFU);
14     SP101_mWriteReg(BASEADDR, REG_BUS_DIN,  data);
15 }
16
17 static u32 mem_read(u32 addr)
18 {
19     SP101_mWriteReg(BASEADDR, REG_CTRL,      CTRL_BUS_EN);
20     SP101_mWriteReg(BASEADDR, REG_BUS_ADDR, addr & 0x3FFU);
21     for (volatile int s = 0; s < 64; s++) { }
22     return SP101_mReadReg(BASEADDR, REG_SP_DOUT);
23 }
24
25 static void processor_reset(void)
26 {
27     SP101_mWriteReg(BASEADDR, REG_CTRL, CTRL_RESET | CTRL_BUS_EN);
28     for (volatile int s = 0; s < 64; s++) { }
29     SP101_mWriteReg(BASEADDR, REG_CTRL, CTRL_BUS_EN);
30 }
31
32 static void processor_run_and_wait(void)
33 {
34     SP101_mWriteReg(BASEADDR, REG_CTRL, CTRL_RUN);
35     while ((SP101_mReadReg(BASEADDR, REG_DONE) & 0x1U) == 0U) { }
36     SP101_mWriteReg(BASEADDR, REG_CTRL, 0U);
37 }
38
39 /* Load 'sc x ssq halt' at memory[0..3]. */
40 static void load_ssq_program(u32 x)
41 {
42     const u32 prog[4] = { OP_SC, x, OP_SSQ, OP_HALT };
43     for (u32 i = 0; i < 4U; i++) {
44         mem_write(i, prog[i]);
45     }
46 }
47
48 /* Per-scenario: reset -> poison mem[128] -> load program -> run ->
49 * read mem[128] -> compare against x*x. */
50 static unsigned run_ssq_scenario(u32 x_in)
51 {
52     const u32 expected = x_in * x_in;
53     const u32 sentinel = 0xDEADBEEFU;
54
55     processor_reset();
56     mem_write(STACK_BASE, sentinel); /* would survive a no-op bug */
57     load_ssq_program(x_in);
58     processor_run_and_wait();
59
60     u32 got = mem_read(STACK_BASE);
61     /* xil_printf("address 128 = %u (expected %u = %u^2) %s",
62     *         got, expected, x_in, (got==expected)?"PASS":"FAIL"); */
63     return (got == expected) ? 0U : 1U;
64 }

```

Listing 3: Core of main.c: bus write, bus read, processor reset, and the per-scenario flow that proves $\text{addr } 128 = x^2$.

8 Part 3.3 – Hardware Results (UART, 4 pts)

The same twelve scenarios were re-executed on the Cora Z7-07S over the AXI4-Lite interface (base 0x43C3_0000). The UART transcript (Figure 7, captured at 115 200 Bd through ps7_uart_0) reports every read-back of mem[128] as PASS – decimal first, hex on the next line – and ends with Summary : 12/12 scenarios PASSED, matching the simulation result line-for-line. Listing 4 reproduces the captured trace verbatim so the prompt’s “address 128 contains the square” requirement is visible in text as well as in the screenshot.

```

=====
ECEC 661 FINAL - Stack Processor 101 + ssq
IP base address : 0x43C30000
=====

---- Scenario: ssq x=7 -> 49 ----
program : sc 7 ssq halt
address 128 = 49 (expected 49 = 7^2) PASS
hex: got=0x00000031 expect=0x00000031

---- Scenario: ssq x=0 -> 0 ----
program : sc 0 ssq halt
address 128 = 0 (expected 0 = 0^2) PASS
hex: got=0x00000000 expect=0x00000000

---- Scenario: ssq x=1 -> 1 ----
program : sc 1 ssq halt
address 128 = 1 (expected 1 = 1^2) PASS
hex: got=0x00000001 expect=0x00000001

---- Scenario: ssq x=2 -> 4 ----
program : sc 2 ssq halt
address 128 = 4 (expected 4 = 2^2) PASS
hex: got=0x00000004 expect=0x00000004

---- Scenario: ssq x=3 -> 9 ----
program : sc 3 ssq halt
address 128 = 9 (expected 9 = 3^2) PASS
hex: got=0x00000009 expect=0x00000009

---- Scenario: ssq x=10 -> 100 ----
program : sc 10 ssq halt
address 128 = 100 (expected 100 = 10^2) PASS
hex: got=0x00000064 expect=0x00000064

---- Scenario: ssq x=15 -> 225 ----
program : sc 15 ssq halt
address 128 = 225 (expected 225 = 15^2) PASS
hex: got=0x000000E1 expect=0x000000E1

---- Scenario: ssq x=16 -> 256 ----
program : sc 16 ssq halt
address 128 = 256 (expected 256 = 16^2) PASS
hex: got=0x00000100 expect=0x00000100

---- Scenario: ssq x=255 -> 65025 ----
program : sc 255 ssq halt
address 128 = 65025 (expected 65025 = 255^2) PASS
hex: got=0x0000FE01 expect=0x0000FE01

---- Scenario: ssq x=1000 -> 1000000 ----
program : sc 1000 ssq halt
address 128 = 1000000 (expected 1000000 = 1000^2) PASS
hex: got=0x000F4240 expect=0x000F4240

---- Scenario: ssq x=32767 -> 1073676289 ----
program : sc 32767 ssq halt
address 128 = 1073676289 (expected 1073676289 = 32767^2) PASS
hex: got=0x3FFF0001 expect=0x3FFF0001

---- Scenario: ssq x=46340 -> 2147395600 ----
program : sc 46340 ssq halt
address 128 = 2147395600 (expected 2147395600 = 46340^2) PASS
hex: got=0x7FFEA810 expect=0x7FFEA810

Summary : 12/12 scenarios PASSED
=====

```

Figure 7: UART trace from the bare-metal sp101 + ssq test app on the Cora Z7-07S. Every scenario shows address 128 = x^2 (decimal) and matching hex on the next line; the summary line at the bottom reads Summary : 12/12 scenarios PASSED. Note the prompt case at the top: address 128 = 49 (expected 49 = 7^2) PASS.

```

1 =====
2 ECEC 661 FINAL - Stack Processor 101 + ssq
3 IP base address : 0x43C30000
4 =====
5
6 ---- Scenario: ssq x=7 -> 49 ----
7 program : sc 7 ssq halt
8 address 128 = 49 (expected 49 = 7^2) PASS
9 hex: got=0x00000031 expect=0x00000031
10
11 ---- Scenario: ssq x=0 -> 0 ---- ... PASS
12 ---- Scenario: ssq x=1 -> 1 ---- ... PASS
13 ---- Scenario: ssq x=2 -> 4 ---- ... PASS
14 ---- Scenario: ssq x=3 -> 9 ---- ... PASS
15 ---- Scenario: ssq x=10 -> 100 ---- ... PASS
16 ---- Scenario: ssq x=15 -> 225 ---- ... PASS
17 ---- Scenario: ssq x=16 -> 256 ---- ... PASS
18 ---- Scenario: ssq x=255 -> 65025 ---- ... PASS
19 ---- Scenario: ssq x=1000 -> 1000000 ---- ... PASS
20 ---- Scenario: ssq x=32767 -> 1073676289 ---- ... PASS
21
22 ---- Scenario: ssq x=46340 -> 2147395600 ----

```



```

23 program : sc 46340 ssq halt
24 address 128 = 2147395600 (expected 2147395600 = 46340^2) PASS
25 hex: got=0x7FFEA810 expect=0x7FFEA810
26
27 -----
28 Summary : 12/12 scenarios PASSED
29 -----

```

Listing 4: Captured UART trace (verbatim from the Cora Z7-07S run; abbreviated – full text in Fig. 7).

9 Conclusion

The Final design adds a 6-state `ssq` micro-program (Listing 1, opcode `0x00000041`) to the assignment PDF’s Stack Processor 101, packages the result as the AXI4-Lite IP `sp101_axi` (Listing 2 for the component declaration and port map required by Part 2), and exercises twelve vectors in both behavioural simulation (Figures 2–6: `checks=12`, `errors=0`) and on the Cora Z7-07S (Figure 7 / Listing 4: `12/12 scenarios PASSED`). For the prompt example `sc 7 ssq halt`, `mem[128] = 49` on both simulation and hardware and `sp = 129` points at the next available address, which is the exact behaviour required by [Final/real/prob.md](#). Each scored deliverable maps to a concrete artifact under [Final/real/stackprocessor_101/](#) and is reproduced step-by-step in [Final/real/README.md](#).